

作業系統期末考 — 歷屆考古題彙整 (2011–2016)

範圍：Ch5 投影片第 16 頁 (CPU 排程演算法 FCFS 之後) → Ch9 虛擬記憶體。題目文字取自原始考卷 PDF (掃描檔以 OCR 還原)，相同題型跨年出現時合併並標註年份。答案請見 [期末考古題解答.md](#)。

壹、CPU 排程 (CPU Scheduling)

【題型 1】★★ 多層反饋佇列甘特圖 (2011 / 2012 / 2013 皆考，必考)

章節 / 主題：Ch5 CPU Scheduling → Multilevel Feedback Queue Scheduling、Preemptive Scheduling、Average Waiting Time

題目 (共同版本)：Scheduling — Consider the following two processes P_1 and P_2 , each of which consists of two CPU bursts and one I/O burst. Assume that P_1 and P_2 arrive at time 0 and 2 (millisecond) respectively.

年份	P_1	P_2
2011	CPU 6 → I/O 4 → CPU 12	CPU 18 (單一 burst)
2012 / 2013	CPU 8 → I/O 4 → CPU 8	CPU 2 → I/O 2 → CPU 16

(a) Draw a Gantt chart illustrating the execution of the processes using the following **preemptive multilevel feedback-queue scheduling algorithm**. Note that **a high-level process will preempt a low-level process** and **a process will be scheduled to a lower level if it is preempted**.

Round-Robin, Time quantum = 4 ms	← 最高層
Round-Robin, Time quantum = 6 ms	← 中層
FCFS scheduling	← 最低層

(b) What is the average waiting time for the scheduling algorithm?

【題型 2】多處理器排程 — 負載平衡 (2015 Q1a 考)

章節 / 主題：Ch5 CPU Scheduling → Multiple-Processor Scheduling、Load Balancing、Push/Pull Migration

題目：Describe two general approaches to **load balancing** of multiple-processor scheduling.

【題型 3】Little's Formula (2015 Q1b 考)

章節 / 主題： Ch5 CPU Scheduling → Algorithm Evaluation、Queueing Models、Little's Formula

題目： What is **Little's formula**? Explain why scheduling analyses based on it may be questionable.

【題型 4】等待時間 vs 完成時間 (2016 Q1a 考)

章節 / 主題： Ch5 CPU Scheduling → Scheduling Criteria、Waiting Time、Turnaround Time

題目： Explain the difference between **waiting time** and **turnaround time**. These times are both used to measure the effectiveness of scheduling schemes.

【題型 5】多層佇列排程 (選擇題 · 2016 Q1b 考)

章節 / 主題： Ch5 CPU Scheduling → Multilevel Queue Scheduling、Scheduling Queues

題目： Which of the following is true of **multilevel queue scheduling**?

- (A) Processes can move between queues.
 - (B) Each queue has its own scheduling algorithm.
 - (C) It is the most general CPU-scheduling algorithm.
 - (D) None of the above.
-

【題型 6】★ 飢餓與防止 (2016 Q1c 考)

章節 / 主題： Ch5 CPU Scheduling → Priority Scheduling、Starvation、Aging

題目： Explain the process of **starvation** and how can we prevent it.

貳、行程同步 (Process Synchronization)

【題型 7】★★ TestAndSet 的 n-process 解 (2011 Q3 考)

章節 / 主題： Ch6 Process Synchronization → Critical-Section Problem、Synchronization Hardware、TestAndSet、Bounded Waiting

題目： With the support of synchronization hardware, solutions of the critical-section problem can be designed in a simple manner. **(a)** Complete the following **n-process** critical-section solution using the **TestAndSet** atomic instruction.

```
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

```
// shared variables 'lock' and 'waiting[n]' are initialized to false
do {
    _____ // ( 填空 )
    CRITICAL SECTION
    _____ // ( 填空 )
    REMAINDER SECTION
} while (TRUE);
```

(b) Show that the code fragments satisfy the three requirements of critical-section solution.

【題型 8】★★ Peterson's Solution 填空 (2012 Q3 考)

章節 / 主題 : Ch6 Process Synchronization → Critical-Section Problem、Peterson's Solution、Mutual Exclusion / Progress / Bounded Waiting

題目 : (a) Complete the following entry and exit code fragments of **Peterson's solution** for process P_0 .

```
// shared data for Peterson's solution
int turn;
boolean flag[2];

do {
    _____ // entry section ( 填空 )
    CRITICAL SECTION
    _____ // exit section ( 填空 )
    REMAINDER SECTION
} while (TRUE);
```

(b) Show that the code fragments satisfy the three requirements of critical-section solution.

【題型 9】★★ Dekker's Solution 證明 (2015 Q2a 考)

章節 / 主題 : Ch6 Process Synchronization → Software Solutions to Critical Section、Dekker's Algorithm、Correctness Proof

題目 : The first known correct software solution to the critical-section problem for two processes was developed by **Dekker**. The two processes, P_0 and P_1 , share the following variables:

```
boolean flag[2]; /* initially false */
int turn;
```

The structure of process P_i ($i == 0$ or 1) is shown below; the other process is P_j ($j == 1$ or 0). **Prove that the algorithm satisfies all three requirements** for the critical-section problem.

```
do {
    flag[i] = true;
    while (flag[j]) {
        if (turn == j) {
            flag[i] = false;
            while (turn == j)
                ; /* do nothing */
            flag[i] = true;
        }
    }
    /* critical section */
    turn = j;
    flag[i] = false;
    /* remainder section */
} while (true);
```

【題型 10】★★ 第一類 Readers-Writers 問題 (2012 Q4 / 2016 Q2a 考)

章節 / 主題：Ch6 Process Synchronization → Classical Synchronization Problems、Readers-Writers Problem、Semaphores

題目：In the **first readers-writers problem**, readers have higher priorities to access a shared object (i.e., file). Design the entry and exit sections of the **reader** processes (2012) / **reader and writer** processes (2016) using the following (shared) data structures:

```
semaphore mutex, wrt;    // initialized to 1
int readcount;           // initialized to 0

// reader's protocol
do {
    _____ // ( 填空 )
    // reading is performed
    _____ // ( 填空 )
} while (TRUE);

// writer's protocol (2016 加考)
do {
    _____ // ( 填空 )
    // writing is performed
    _____ // ( 填空 )
} while (TRUE);
```

【題型 11】★★ Producer-Consumer with Pthreads (2013 Q3 考)

章節 / 主題：Ch6 Process Synchronization → Bounded-Buffer / Producer-Consumer Problem、Mutex Locks、Counting Semaphores、Pthreads

題目 (15 分) : Producer-Consumer Problem — Complete the following producer thread using **Pthreads mutex locks and semaphores**.

```
// global (share) variables
int buffer[5];           // shared buffer
pthread_mutex_t mutex;   // mutex for shared data access
sem_t empty;             // the number of empty buffers
sem_t full;              // the number of full buffers
int in, out;             // in: next free position; out: first full position

int main(void) {
    // Initialize mutex and counting semaphores empty and full
    /* Initialize buffer */
    /* Create producer thread(s) */
    /* Create consumer thread(s) */
    /* Sleep */ /* Exit */
}

void *producer(void *param) {
    int random_item;      // the produced item
    int sleep_interval;   // the time of thread sleeping
    while (TRUE) {
        sleep_interval = rand() % 5;
        sleep(sleep_interval); // sleep for a rand period of time
        random_item = rand();  // generate a random integer
        // insert the random_item into the shared buffer ← 填空
    }
}
```

【題型 12】★ Semaphore wait()/signal() 原子性 (2013 Q4 考)

章節 / 主題 : Ch6 Process Synchronization → Semaphores、Atomic Operations、Mutual Exclusion

題目 : The `wait()` and `signal()` methods of a semaphore S are defined as follows:

```
wait(S) {
    while (S <= 0)
        ; // no-operation
    S--;
}

signal(S) {
    S++;
}
```

In `wait()`, the testing of S (i.e., `S <= 0`) and its possible modifications (`S--`) must be **atomic** (executed without interruption). **Use an execution sequence to show that if `S--` is not atomic, then the mutual exclusion of critical section cannot be preserved.** The initial value of S is 1. Hint: `S--` may be implemented in machine language as:

```
register1 = S
register1 = register1 - 1
S = register1
```

【題型 13】臨界區段三需求 / Exit Section 問題 (2014 考)

章節 / 主題：Ch6 Process Synchronization → Critical-Section Problem、Three Requirements、Exit Section Design

題目：(b) Show your solution satisfies the three requirements of the critical-section solution. (c) What is the problem if we simply release the lock in the **exit section**?

【題型 14】Monitor 條件變數訊號模型 (2015 Q2b 考)

章節 / 主題：Ch6 Process Synchronization → Monitors、Condition Variables、Signal-and-Wait / Signal-and-Continue

題目：Regarding **monitor**, what are the two common **signal models of condition variables**? Explain the difference between them.

【題型 15】Counting Semaphore (選擇題 , 2016 Q2b 考)

章節 / 主題：Ch6 Process Synchronization → Semaphores、Counting Semaphore、Critical-Section Control

題目：A **counting semaphore** ____.

- (A) is essentially an integer variable
- (B) is accessed through only one standard operation
- (C) can be modified simultaneously by multiple threads
- (D) cannot be used to control access to a thread's critical sections

參、死結 (Deadlocks)

【題型 16】★★ 銀行家演算法 — 請求是否可批准 (2011 / 2012 / 2014 / 2015 皆考 , 必考)

章節 / 主題：Ch7 Deadlocks → Deadlock Avoidance、Safe State、Banker's Algorithm、Resource-Request Algorithm

題目 (共同版本)：Consider the following snapshot of a **SAFE** system; can a request **(0,4,2,0)** from p_1 be granted? Show your decision by using the **banker's algorithm**. (4 種資源 A,B,C,D, 5 個行程)

	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	0	1	2	0	0	1	2	1	5	2	0
P1	1	0	0	0	1	7	5	0				
P2	1	3	5	4	2	3	5	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

註：2011 / 2012 / 2014 / 2015 為完全相同的 4 資源快照 (如上) · 請求皆為 (0,4,2,0) from p₁。

【題型 17】★★ 銀行家演算法 — 證明請求導致 Unsafe (2013 Q5 考)

章節 / 主題：Ch7 Deadlocks → Deadlock Avoidance、Unsafe State、Banker's Algorithm、Safety Algorithm

題目：(a) Consider the following snapshot of a **SAFE** system; **show why granting request (3,2,1) from p₀ will make the system unsafe** using the banker's algorithm. (3 資源 A,B,C)

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	8	5	3	3	3	2
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

(b) Explain why a deadlock **MAY NOT** occur even if we grant the above request. And under which condition, a deadlock occurs.

【題型 18】★ Unsafe 能否回到 Safe (2011 / 2012 / 2014 考)

章節 / 主題：Ch7 Deadlocks → Safe / Unsafe State、Deadlock Avoidance Concepts

題目：Can an **UNSAFE** system back to a **SAFE** state? Explain your answer.

【題型 19】★ Unsafe 不一定造成 Deadlock (2015 Q3b / 2016 Q3a 考)

章節 / 主題：Ch7 Deadlocks → Relationship between Unsafe State and Deadlock、Deadlock Avoidance

題目：Explain why an unsafe state **MAY BUT NOT NECESSARILY** lead to a deadlock.

【題型 20】★ 死結的四個必要條件 (2012 / 2014 / 2015 考)

章節 / 主題：Ch7 Deadlocks → Deadlock Characterization、Four Necessary Conditions

題目：Explain the **four necessary conditions** of a deadlock.

【題型 21】★ 證明資源排序可防止 Circular Wait (2016 Q3b 考)

章節 / 主題：Ch7 Deadlocks → Deadlock Prevention、Circular Wait、Resource Ordering

題目：A method to prevent deadlock is to impose a **total ordering of all resource types**. And each process can request resources only in an increasing order of enumeration. **Proof that the circular-wait condition cannot hold if the method is employed.**

【題型 22】m 資源 n 行程的無死結證明 (2014 Q3 考)

章節 / 主題：Ch7 Deadlocks → Deadlock-Free Resource Allocation、Resource Instances、Deadlock Proof

題目 (10 分)：Deadlocks — consider a system consisting of **m** resources of the same type, being shared by **n** processes. Resources can be requested and released by processes only one at a time. Show that the system is **deadlock free** if the following two conditions hold:

- The maximum need of each process is between 1 and m resources.
- The sum of all maximum needs is less than $m + n$.

【題型 23】Safe/Unsafe 選擇題 (2016 Q3c 考)

章節 / 主題：Ch7 Deadlocks → Safe State、Unsafe State、Deadlocked State

題目：Which of the following statements is true?

- (A) A safe state is a deadlocked state.
- (B) A safe state may lead to a deadlocked state.
- (C) An unsafe state is necessarily, and by definition, always a deadlocked state.
- (D) An unsafe state may lead to a deadlocked state.

肆、記憶體管理 (Main Memory)

【題型 24】★★ 分頁系統 — 頁表大小計算 (2011 / 2012 / 2013 / 2014 / 2015 / 2016 皆考，必考)

章節 / 主題：Ch8 Main Memory → Paging、Page Table Size、Logical Address、Page Offset

題目 (共同版本)：Consider a paging system that supports **N-bit logical address** and has **M bytes physical memory**. The size of a page is 4 bytes. **(a)** If the size of a page table entry is **2 bytes** (first byte for frame number & second byte for protection bits), then **how many bytes are required for a page table?**

年份	logical address	physical memory	page size
2011 / 2013 / 2014 / 2015	4-bit	32 bytes	4 bytes
2012	4-bit	64 bytes	4 bytes

年份	logical address	physical memory	page size
2016	5-bit	64 bytes	4 bytes

【題型 25】★★ 多層分頁 — 填寫邏輯內容 (2011 / 2012 / 2013 / 2014 / 2015 / 2016 皆考，必考)

章節 / 主題： Ch8 Main Memory → Hierarchical Paging、Two-Level / Three-Level Paging、Page Tables、Frame Mapping

題目 (共同版本)： The following figure shows the physical memory content and the **outer page table** (two-level paging) / **3rd page table** (three-level paging) of a process. **Fill the logical content of the process.** (給定 register-based outer/3rd page table · 需用 frame ID + protection/valid bit 反推 logical address 的內容)

- 2011 / 2014 / 2015： **two-level paging**, outer page table = {0→frame 4, 1→frame 2}
- 2012 / 2016： **three-level paging**, 3rd page table = {0→frame 4, 1→frame 2}
- 2013 / 2015： 另有「**allocate memory (frames) for the process and the page table**」變體 · free-frame list = 5, 0, 2, 7, 3, 4, Frame 5 為第一個 free frame；先配置頁表再配置行程

【題型 26】Address Binding 三時機 (2015 Q4 考)

章節 / 主題： Ch8 Main Memory → Address Binding、Compile-Time / Load-Time / Execution-Time Binding、Relocation

題目 (10 分)： Technically, the binding of instructions and data to physical memory addresses can be done at **compile time**, **load time**, or **execution time**. Explain the three binding approaches and **show the weakness of the first two**.

【題型 27】Compaction (選擇題 · 2016 Q4a 考)

章節 / 主題： Ch8 Main Memory → Contiguous Memory Allocation、External Fragmentation、Compaction、Dynamic Relocation

題目： Which of the following is true of **compaction**?

- (A) It can be done at assembly, load, or execution time.
- (B) It is used to solve the problem of internal fragmentation.
- (C) It is possible only if relocation is dynamic and done at execution time.
- (D) None of the above.

伍、虛擬記憶體 (Virtual Memory)

【題型 28】★★ 頁面置換 — 計算 page fault 數 (2013 / 2014 / 2016 皆考，必考)

章節 / 主題：Ch9 Virtual Memory → Page Replacement、Page Faults、Optimal Replacement、LRU Replacement

題目 (2014 版本)：Virtual Memory — Consider the following page reference string: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6. How many page faults would occur for the following replacement algorithms, assuming **four** frames are available?

- LRU replacement
- Optimal replacement

題目 (2013 / 2016 版本)：Consider the following logical memory references to a system with **2KB** physical memory which is partitioned into **4 frames** of the same size. How many page faults would occur for **(5 points) the optimal** page-replace algorithm and **(5 points) the LRU** page replacement algorithm? (2013/2016 給定一串 9-bit 二進位邏輯位址，需先換算成 page number)

【題型 29】★ Page Fault 處理流程 (2013 Q7a / 2015 Q5b / 2016 Q5a 考)

章節 / 主題：Ch9 Virtual Memory → Demand Paging、Page-Fault Handling、Valid-Invalid Bit、Modify Bit、Page Replacement

題目 (2013 / 2016)：Describe the actions taken by the operating system when a user process tries to access an **invalid memory page**. You need to consider whether the reference is **invalid or it is a page fault**. You also need to consider **page replacement** when there is no free frame for a page fault. In addition, a **modify bit** is provided to improve page-fault service time.

題目 (2015 Q5b)：The following figure shows the procedure for handling a page fault. Explain the function of **steps 1 to 6**. Here, we assume a free frame is always available.

【題型 30】虛擬記憶體優於分頁 (2015 Q5a 考)

章節 / 主題：Ch9 Virtual Memory → Virtual Memory Benefits、Demand Paging、Logical Address Space、Process Creation

題目：Show **two advantages of virtual memory over paging**.

附錄：考試年份題目對照索引

主題	2011	2012	2013	2014	2015	2016
多層反饋佇列甘特圖	Q2	Q2	Q2	—	—	—
多處理器 / Little's	—	—	—	—	Q1	—
等待 vs 完成時間	—	—	—	—	—	Q1a
多層佇列 (選擇)	—	—	—	—	—	Q1b

主題	2011	2012	2013	2014	2015	2016
飢餓與防止	—	—	—	—	—	Q1c
TestAndSet n-process	Q3	—	—	—	—	—
Peterson's 填空	—	Q3	—	—	—	—
Dekker's 證明	—	—	—	—	Q2a	—
Readers-Writers	—	Q4	—	—	—	Q2a
Producer-Consumer	—	—	Q3	—	—	—
Semaphore 原子性	—	—	Q4	—	—	—
Monitor 訊號模型	—	—	—	—	Q2b	—
Counting semaphore (選擇)	—	—	—	—	—	Q2b
銀行家 (可否批准)	Q4	Q5	—	Q4	Q3	—
銀行家 (證明 unsafe)	—	—	Q5	—	—	—
Unsafe→Safe	Q4a	Q5b	—	Q4b	—	—
Unsafe 不一定死結	—	—	Q5b	—	Q3b	Q3a
死結四條件	—	Q5a	—	Q4a	Q3a	—
資源排序防 circular wait	—	—	—	—	—	Q3b
m 資源 n 行程證明	—	—	—	Q3	—	—
頁表大小計算	Q5a	Q6a	Q6a	Q5a	Q6a	Q4b
多層分頁填內容	Q5b	Q6b	Q6b	Q5b	Q6b	Q4c
Address Binding	—	—	—	—	Q4	—
Compaction (選擇)	—	—	—	—	—	Q4a
頁面置換計算	—	—	Q7b	Q6	—	Q5b
Page Fault 流程	—	—	Q7a	—	Q5b	Q5a
VM 優於分頁	—	—	—	—	Q5a	—
名詞解釋	Q1	Q1	—	—	—	—

名詞解釋彙整：

- 2011 Q1：Turnaround time、Race condition、Deadlock、External fragmentation、Demand paging
- 2012 Q1：Processor affinity、Critical section、Safe sequence of processes、Worst-fit strategy of contiguous memory allocation、Inverted page table

End of 期末考古題彙整。